



Eötvös Loránd University

Faculty of Informatics

Department of Artificial Intelligence

A Tool to Study Swarm Intelligence Algorithms Benefiting from GPU Acceleration

Supervisor:

László Gulyás

Associate professor

Author:

Supawarapong Ryan

Computer Science BSc

Budapest, 2025

Thesis Topic Declaration

In this thesis, I aim to explore the capabilities and potential of agent-based simulations using FLAME GPU 2 platform to experiment on collective behaviors and swarm intelligence problems. Swarm intelligence is the remarkable phenomenon where collective behavior naturally arises from a group of simple agents adhering to fundamental rules.

FLAME GPU is a GPU accelerated agent-based simulation library for domain independent complex systems simulations. It provides a mapping between a formal agent specifications with C++ based scripting and optimized CUDA code. This includes a number of key Agent-based Modelling (ABM) building blocks such as multiple agent types, agent communication and birth and death allocation.

The goal of this thesis is to develop a program to simulate various swarm intelligence algorithms, such as particle swarm optimization, ant colony optimization, and boids flocking behavior, using the FLAME GPU 2 platform. The program will be designed to allow for extensive experimentation with these algorithms by providing functionality to study various parameter settings, including the ability to handle a large number of agents. By capitalizing on GPU acceleration, the program aims to demonstrate enhanced computational efficiency and scalability.

Table of Contents

Introduction	1
User Documentation.....	2
Requirements of the software version	2
Hardware Requirements	2
Installation Guide	3
1. Install Python and Cuda toolkit.....	3
2. Clone the Repository and Git.....	4
3. Create Virtual Environment	5
4. Install FLAME GPU 2 and other package	5
5. Running the Program	6
Developer Documentation	16
1. Requirement Analysis.....	16
Functional Requirement.....	16
Non-Functional Requirements	16
Use Case diagrams	17
2. Architecture of the program.....	17
2.1 User Interface Layer.....	18
2.2 Simulation Logic Layer	19
2.3 Core Simulation Engine.....	19
3.1 Class MainApp.....	19
3.2 Class ParticleSwarm	20
3.3 Class Boids2D.....	21
3.4 Class AntColony	22

4. Implementation Notes.....	23
4.1 Technical Challenges and Solutions.....	23
4.2 Development Tools and Technologies	23
5. Testing Plan	24
5.1 Unit Testing.....	24
5.2 Integration Testing	25
5.3 Performance Testing.....	25
6. Testing.....	27
6.1 Unit Testing Results	27
6.3 Integration Testing Results.....	28
Summary	29
References	30

Chapter 1

Introduction

Swarm intelligence algorithms, such as Boids (Reynolds, 1987), Ant Colony Optimization(ACO) (Dorigo & Gambardella, 1997), Particle Swarm Optimization(PSO), are computational methods that optimize a problem by iteratively trying to solve complex optimization problems. However, simulating these algorithms can be computationally expensive and slow when dealing with a large number of agents on a CPU.

To address this challenge, this thesis explores the potential of GPU acceleration to enhance the performance of swarm intelligence algorithms. This project utilizes FLAME GPU 2 (Richmond, Chimeh, Chisholm, Heywood, & Leach, FLAME GPU (v2.0.0-rc.2), 2024), a framework designed for agent-based modeling on GPU with CUDA support, to demonstrate the benefits of GPU acceleration in scaling ABM to large numbers of agents.

An agent-based model is a computational model that simulates the actions and interactions of autonomous agents based on predefined rules.

Chapter 1

User Documentation

This software addresses the problem of simulating and analyzing collective behaviors through agent-based modelling at a large scale, specifically using Boids, Particle Swarm Optimization (PSO) algorithm and Ant Colony Optimization (ACO) algorithm (Kennedy & Eberhart, 1995) (Shi & Eberhart, 1998). This software leverages GPU acceleration to enable more efficient and scalable simulations compared to CPU-based simulation.

FLAME GPU 2 is responsible for the execution, management, and acceleration of agent-based models (ABMs).

Requirements of the software version

- Python 3 version: 3.8 up to 3.12 are officially supported by FLAME GPU 2.
- FLAME GPU 2 version: 2.0.0-rc.2 (Richmond, Chimeh, Chisholm, Heywood, & Leach, FLAME GPU (v2.0.0-rc.2), 2024)
- CUDA toolkit version: 11.0 up to 12.x.
- Operating System: Ubuntu 24.04.2 LTS/Window 10 or 11

Hardware Requirements

- CPU: X86_64
- GPU: Nvidia GPU with Compute Capability 3.5 or more.

Installation Guide

1. Install Python and Cuda toolkit

- Window 10/11: You will need Python version 3.12.x from the official Python website at <https://www.python.org/>. Navigate to find older release of Python 3. We will install the latest available version which is Python 3.12.10 at <https://www.python.org/downloads/release/python-31210/>. Install Python to the system by following the installer instructions. Make sure to add python to PATH during the installation.
- Linux (Ubuntu 24.04.2 LTS): Python 3.12 should already be installed by default. If Python is not installed, you can install it with “`sudo apt install python3`” for Debian based distro as demonstrated in the Figure 1.

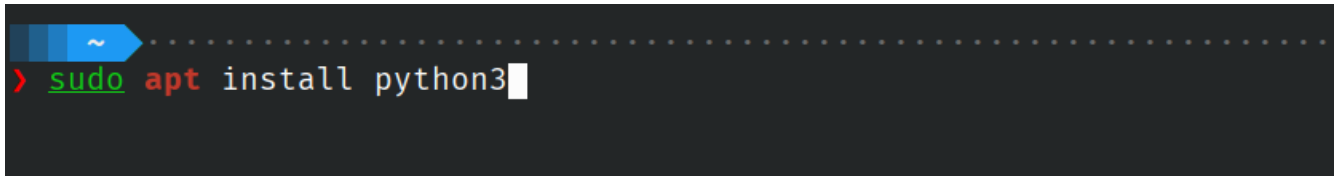
A terminal window with a dark background. The prompt is a blue arrow pointing right with a tilde (~) inside. The command `sudo apt install python3` is entered in green text, followed by a white cursor.

Figure 1

You will also need to install venv with “`sudo apt install python3.12-venv`” to be able to use virtual environment as shown in the Figure 2.

A terminal window with a dark background. The prompt is a blue arrow pointing right with a tilde (~) and the text `~/swarm-intelligent` in white, followed by a green arrow pointing right with the text `main` in white. The command `sudo apt install python3.12-venv` is entered in green text, followed by a white cursor.

Figure 2

After that you must install CUDA toolkit from the <https://developer.nvidia.com/cuda-downloads/>. For example, on Ubuntu 24.04.2 LTS the instruction would be as shown below in Figure 3.

```

> wget https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2404/x86_64/cuda-keyring_1.1-1_all.deb
> sudo dpkg -i cuda-keyring_1.1-1_all.deb
> sudo apt-get update
> sudo apt-get -y install cuda-toolkit-12-8

```

Figure 3

2. Clone the Repository and Git

Open a terminal in Ubuntu or command prompt in window 10/11.

Clone the repository of the project using git or download the project as a zip file from <https://github.com/Max4king/swarm-intelligent/>. Note that Git can be installed from <https://git-scm.com/downloads/> or “sudo apt install git” for Ubuntu shown in Figure 4.

```

> sudo apt install git

```

Figure 4

Once installed, use “git clone <https://github.com/Max4king/swarm-intelligent>” to clone the repository as shown in the Figure 5.

```

> git clone https://github.com/Max4king/swarm-intelligent.git
Cloning into 'swarm-intelligent' ...
remote: Enumerating objects: 8, done.
remote: Counting objects: 100% (8/8), done.
remote: Compressing objects: 100% (7/7), done.
Receiving objects: 100% (8/8), 10.19 KiB | 10.19 MiB/s, done.
Resolving deltas: 100% (1/1), done.
remote: Total 8 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)

```

Figure 5

3. Create Virtual Environment

Go into the directory with “`cd swarm-intelligent`”. Then create a virtual environment for python by running “`python3 -m venv venv`” as shown in the Figure 6.



```
~  
> cd swarm-intelligent  
~/swarm-intelligent main  
> python3 -m venv venv
```

Figure 6

4. Install FLAME GPU 2 and other package

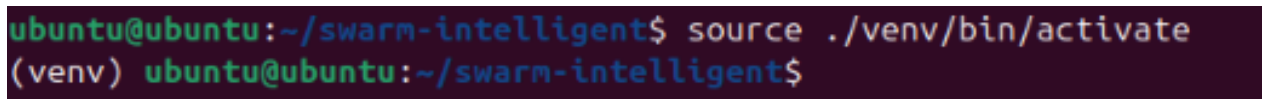
Make sure to activate the virtual environment before installing the package with “`source venv/bin/activate`” for Linux and “`venv\Scripts\activate.bat`” for window on cmd.



```
~/swarm-intelligent main  
> source ./venv/bin/activate  
~/swarm-intelligent main  
>
```

Figure 7

Now you should see the `venv` before your username in the terminal as shown in Figure 8. This indicates that you have successfully activated the virtual environment. This means that you are in the virtual environment that was created earlier.



```
ubuntu@ubuntu:~/swarm-intelligent$ source ./venv/bin/activate  
(venv) ubuntu@ubuntu:~/swarm-intelligent$
```

Figure 8

Next, we can install all the required package with “`pip install -r requirements.txt`”. This will install all the packages needed to run the program as shown in Figure 9.

```

~/swarm-intelligent main
> pip install -r requirements.txt
Looking in indexes: https://pypi.org/simple, https://whl.flamegpu.com/whl/cuda120-vis/
Collecting pyflamegpu=2.0.0rc2 (from -r requirements.txt (line 2))
  Downloading https://github.com/FLAMEGPU/FLAMEGPU2/releases/download/v2.0.0-rc.2/pyflamegpu-2.0.0rc2%2Bcuda120.vis-cp
    174.2/174.2 MB 310.1 kB/s eta 0:00:00
Collecting matplotlib=3.10.1 (from -r requirements.txt (line 3))

```

Figure 9

5. Running the Program

The program can be run with “python main_ui.py”. In case of hybrid graphics setup, we may need to “__NV_PRIME_RENDER_OFFLOAD=1 __GLX_VENDOR_LIBRARY_NAME=nvidia” as environmental variable in order to use the Nvidia GPU properly when enabling visualization as shown in Figure 10.

```

~/swarm-intelligent main
> __NV_PRIME_RENDER_OFFLOAD=1 __GLX_VENDOR_LIBRARY_NAME=nvidia python main_ui.py

```

Figure 10

Once the shown in Figure 11 below appears, you can start choosing the simulation to run.

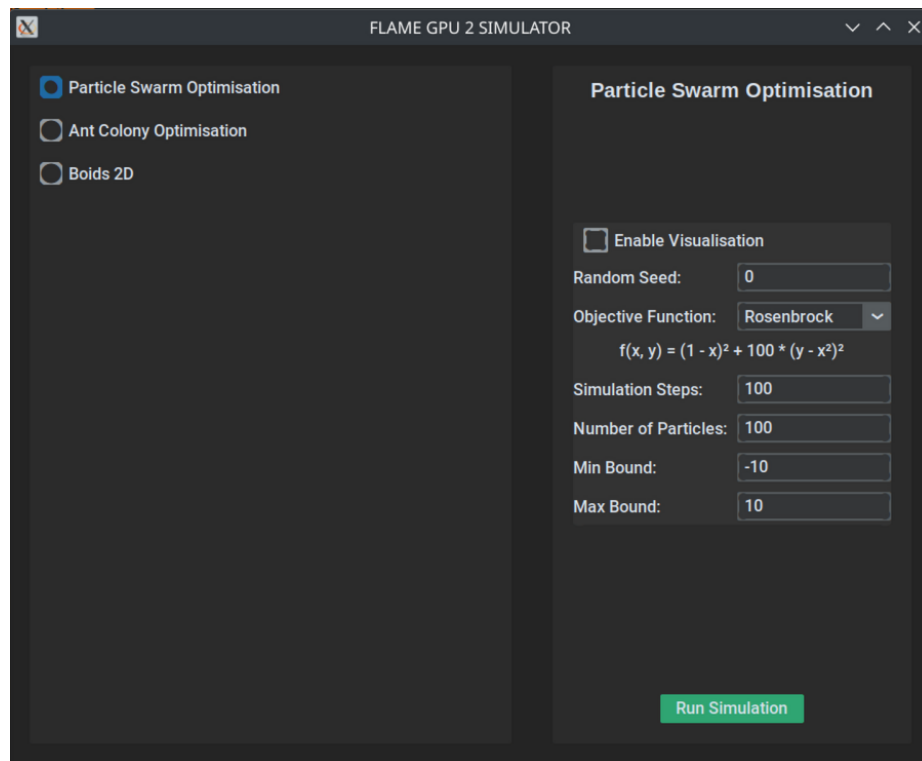


Figure 11

In every algorithm, there are common options such as random number and visualization as shown in Figure 12. When visualization is enabled, it will give the option on the visualization speed as highlighted in Figure 12.

Particle Swarm Optimisation

☒ Enable Visualisation

Random Seed:

Objective Function: Rosenbrock ▼

$f(x, y) = (1 - x)^2 + 100 * (y - x^2)^2$

Simulation Steps:

Number of Particles:

Min Bound:

Max Bound:

Simulation Speed(Visual):

Figure 12

On the other hand, random seeds allow for the simulation to be reproducible. By setting the seed to value 0, a random seed will be generated instead. This random seed is used for all the randomness in the simulation such as initial position to choosing a path.

5.1 Running Particle Swarm Optimization

Figure 13 displays the parameters available for the Particle Swarm Optimization (PSO) algorithm:

- Objective Function: A problem which the PSO will optimize for. The list includes Rosenbrock, Sphere, Booth, Beale and Matyas function. The function formula is displayed below the dropdown.

- Simulation Steps: This parameter determines the total number of steps the simulation will execute.
- Number of particles: The number of particles that will be exploring in the search space.
- Minimum Bound: This is the minimum limit which the particle's starting positions in the search space.
- Maximum Bound: This is the maximum limit which the particle's starting positions in the search space.
- Inertia Weight (w): This parameter controls how much the particle's previous velocity influences its current velocity
- Cognitive Coefficient($c1$): This parameter scales the influence of the particle's own best-known position (pbest).
- Social Coefficient: ($c2$): This parameter scales the influence of the swarm's overall best-known position (gbest).

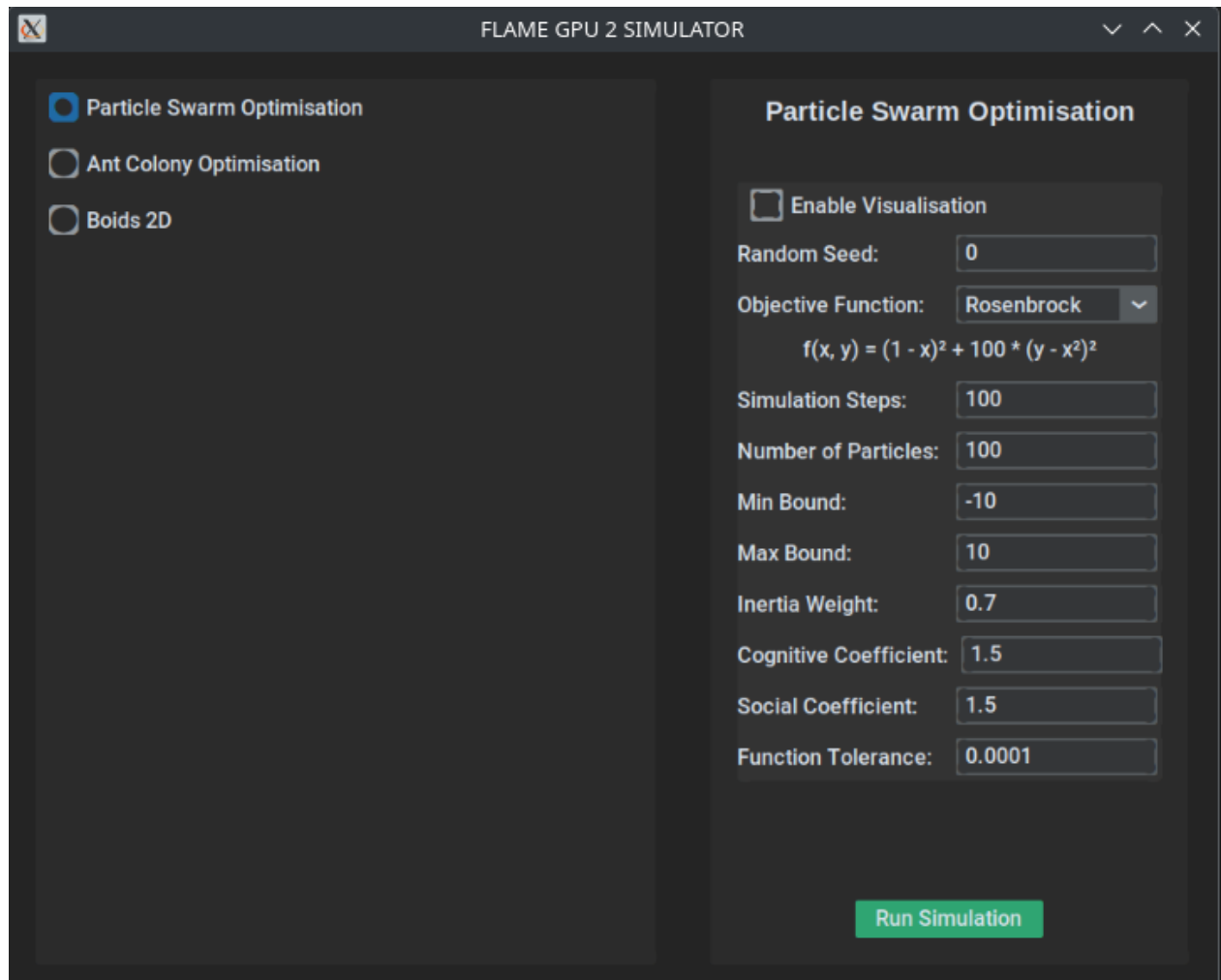


Figure 13

Within the visualization shown in the Figure 15, you can dynamically adjust the w , $c1$, and $c2$ values while the simulation is active to observe the impact of each setting as shown in the Figure 14.

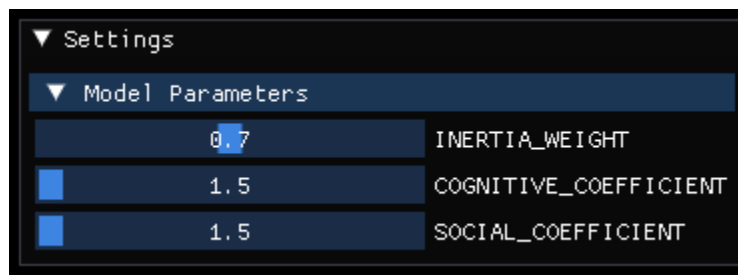


Figure 14

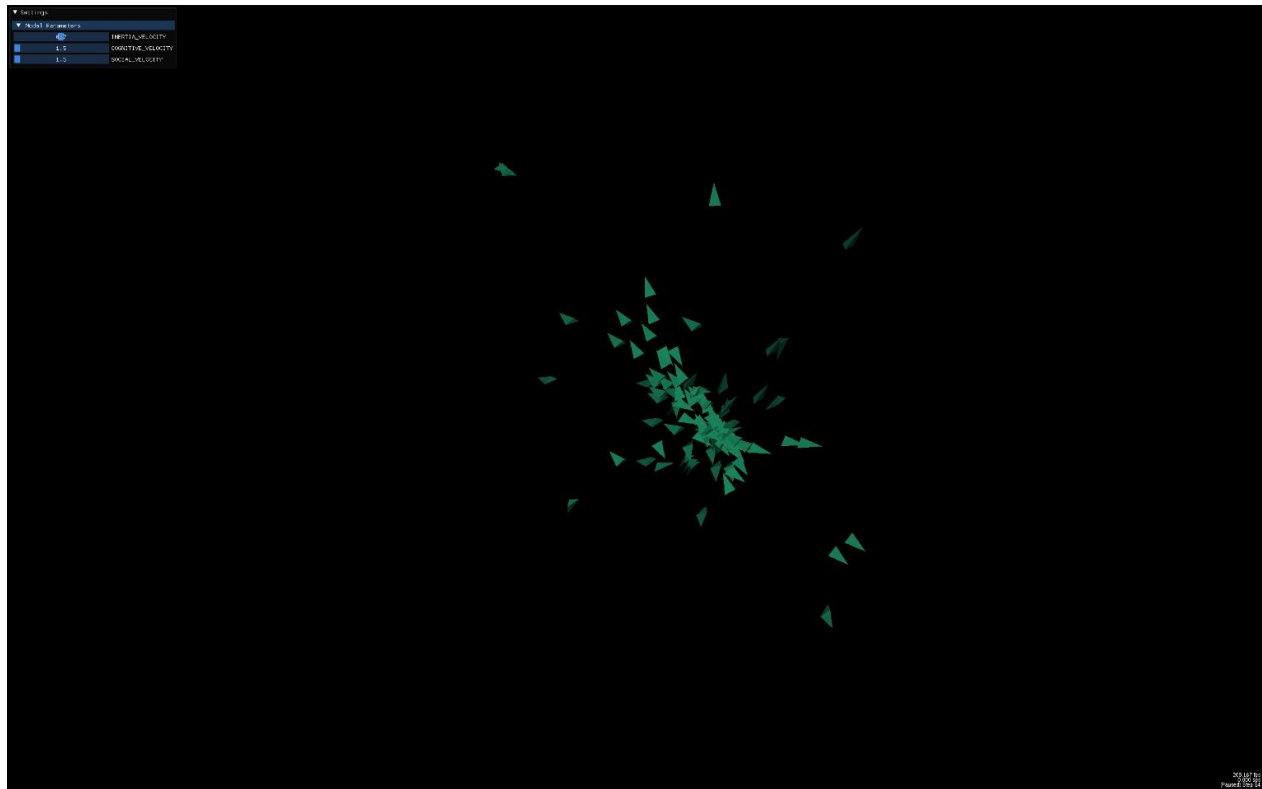


Figure 15

After the simulation, the program will display a new window as shown in the Figure 16 showing the optimal cost and position.

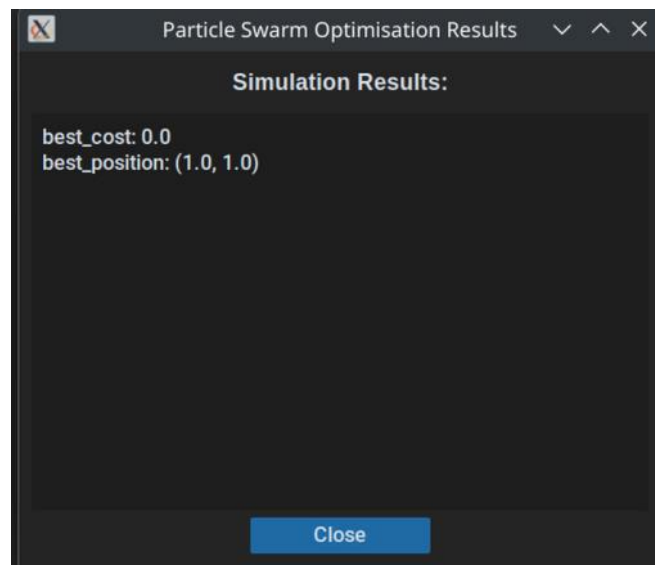


Figure 16

5.2 Running Ant Colony Optimization

The graph will be randomly generated based on the random seed.

Figure 17 shows the parameters available for Ant Colony Optimization (ACO) algorithm:

- Number of Ants: The number of ants that would be exploring the graph for good path.
- Number of Iterations: The number of times the optimization will run for. Each iteration is complete once all the ants are done.
- Grid Size: The range in which the vertices will be positioned. For example, if the grid size is 10, the vertices will be positioned somewhere within a 10x10 area.
- Vertices Count: The number of points in the graph.
- Pheromone Evaporation:
- Alpha: This value controls the influence of pheromone on the ants' path decisions. A higher alpha encourages exploitation of paths with higher pheromone levels, indicating previous success.
- Beta: This value controls the influence of path cost (distance) on the ants' path decisions. A higher beta encourages exploration of shorter paths, potentially discovering new, efficient routes.

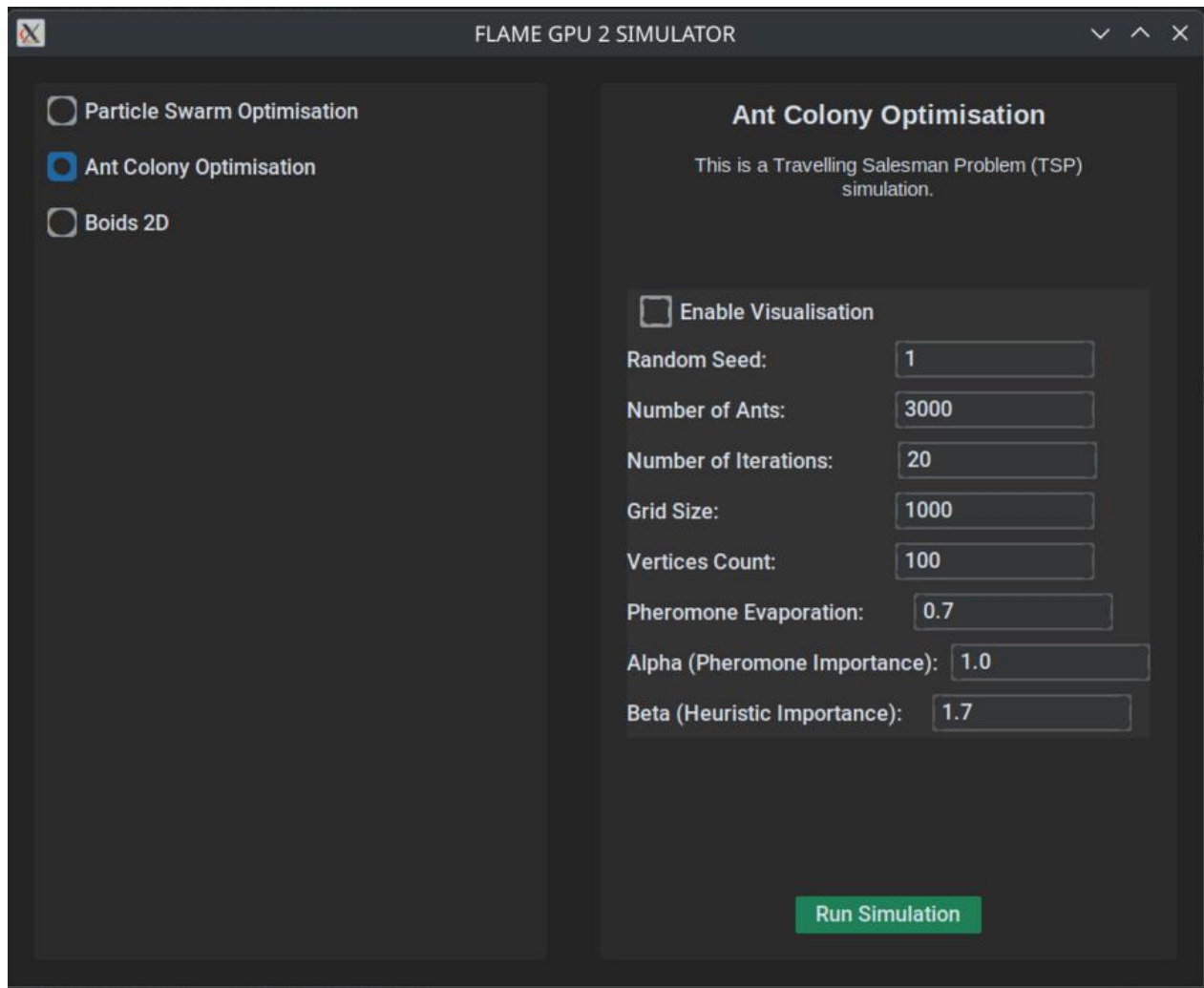


Figure 17

Upon the simulation being done, the result will be displayed in a new window as shown in the Figure 18. We will be able to see the best cost, best path and the total distance.

The best cost represents the lowest total distance of a path discovered by the colony. This best path traverses all vertices and returns to the starting point, with the total distance being the sum of all segment lengths along that route.

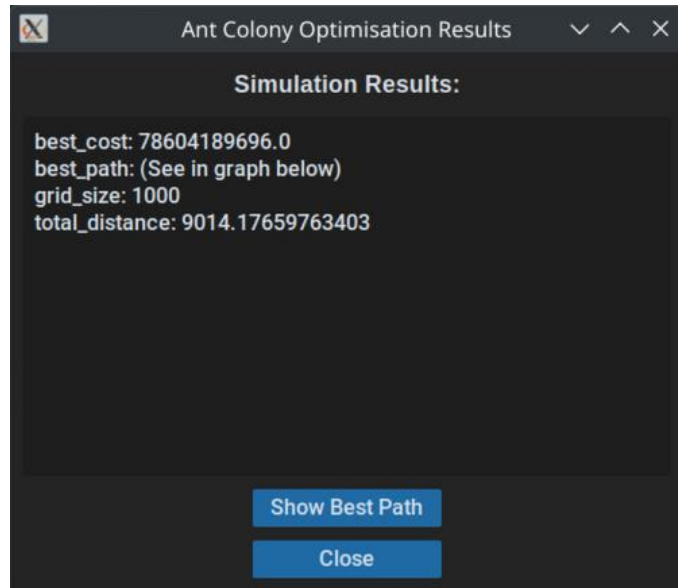


Figure 18

We can also visualize the path by clicking “show Best Path” which will open a separate window to display the figure in a scatter graph with annotation of the best path found by the colony as shown in the Figure 19.

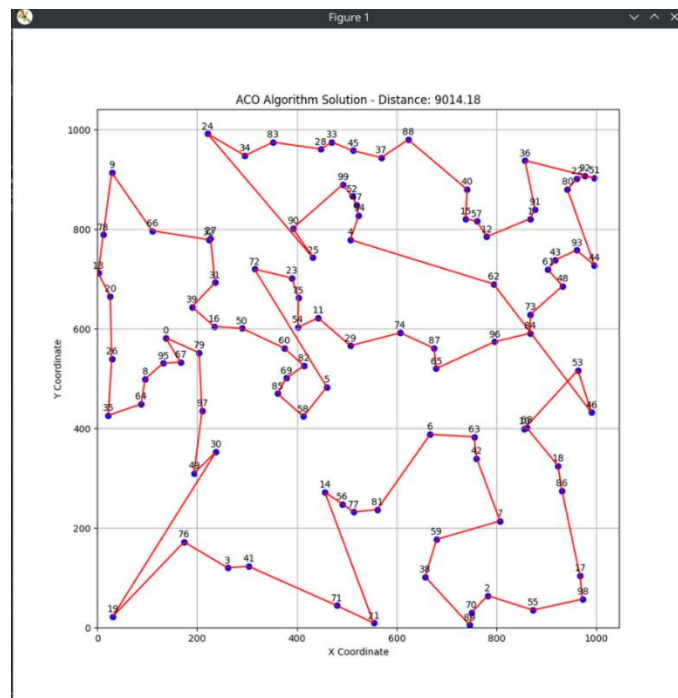


Figure 19

5.3 *Running Boids*

Figure 20 shows the parameters available for Boids:

- Number of Agents: The total count of Boids presents in the simulation.
- Grid Size: Defines the boundaries of the area where Boids can move.
- Acceleration: Controls how rapidly a Boid can change its speed in each step of the simulation.
- Separation Radius: The minimum distance Boids will maintain to avoid clashing and crowding each other.
- Alignment Weight: Adjusts the strength of the force that encourages a Boid to match the average direction of its nearby neighbors.
- Cohesion Weight: Adjusts the strength of the force that pulls a Boid towards the perceived center of the group of its nearby neighbors.
- Separation Weight: Adjusts the strength of the force that pushes a Boid away from its nearby neighbors
- Max Steer Force: Determines the maximum amount a Boid can change its direction in a single simulation step.
- Max Speed: Caps the maximum speed a Boid could go.

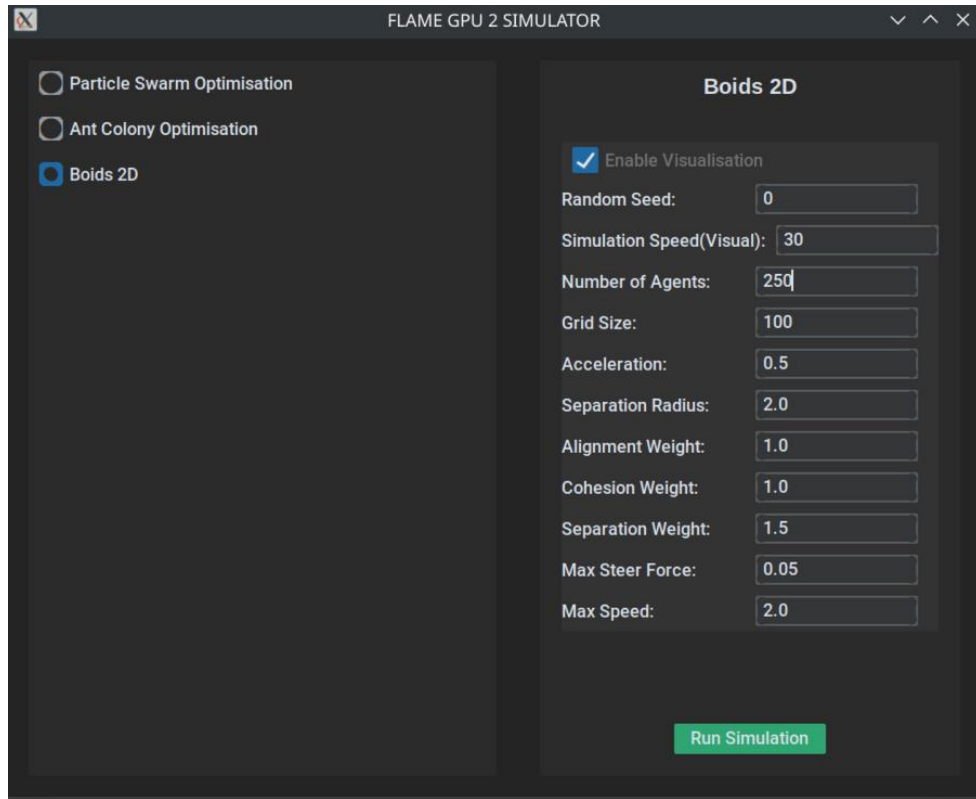


Figure 20

In the visualization, you can control the weight, speed, acceleration and separation radius of the Boids while the simulation is running to see the effect real time as shown in the Figure 21.

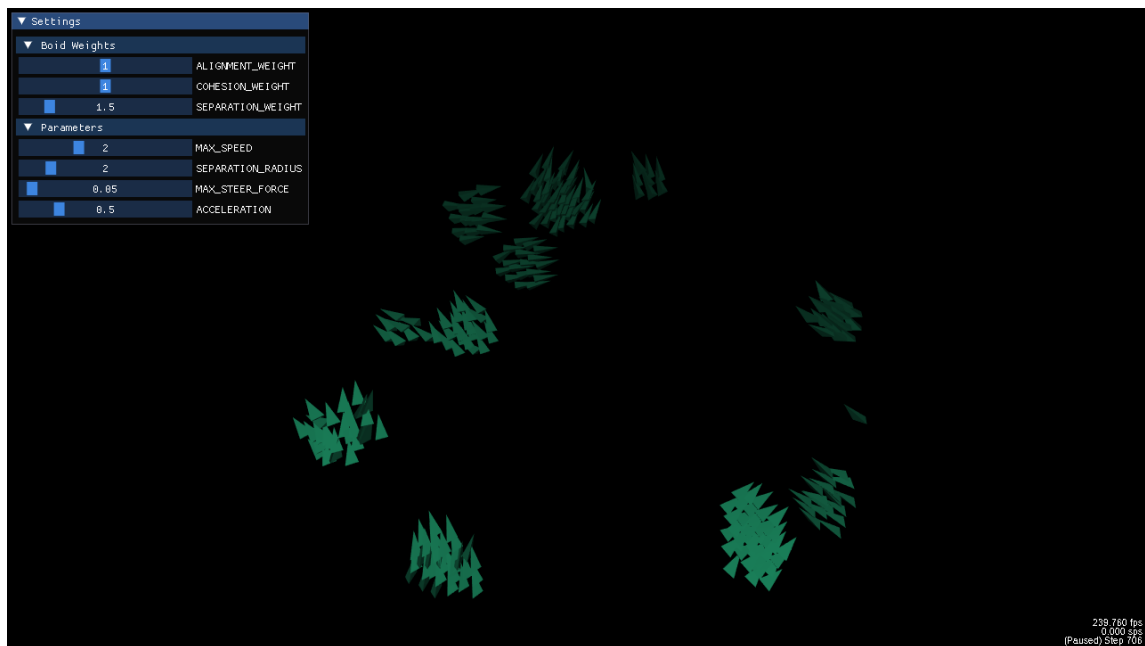


Figure 21

Chapter 3

Developer Documentation

The goal is to develop a tool which allows the user to study and visualize swarm intelligence algorithms using GPU acceleration with FLAMEGPU 2. This will allow better performance more efficiently with parallel processing of a GPU. To overcome this limitation, GPU are used to accelerate and increase scalability of the simulation.

1. Requirement Analysis

Functional Requirement

- Simulate Particle Swarm Optimization (PSO): The program must be able to execute simulations based on the PSO algorithm using the FLAME GPU 2 framework.
- Simulate Ant Colony Optimization (ACO): The program must be able to execute simulations to optimize for TSO based on the ACO algorithm using the FLAME GPU 2 framework.
- Simulate Boids flocking behavior: The program must be able to execute simulations based on the Boids flocking behavior using the FLAME GPU 2 framework.
- Utilize FLAME GPU 2: Must use FLAME GPU 2 library for GPU acceleration.
- Agent Capacity: The program must be able to handle simulating large number of agents.

Non-Functional Requirements

- Usability: The interface or method for configuring parameters and running experiments should be straightforward .
- Performance: The program should scale efficiently with large numbers of agents in a simulation without prohibitive increases in execution time.
- Correctness: The implementations of PSO, ACO, and Boids must be accurate according to its respective rules and formula.

Use Case diagrams

This use case diagrams illustrates the interaction between the user and the program as shown in Figure 22.

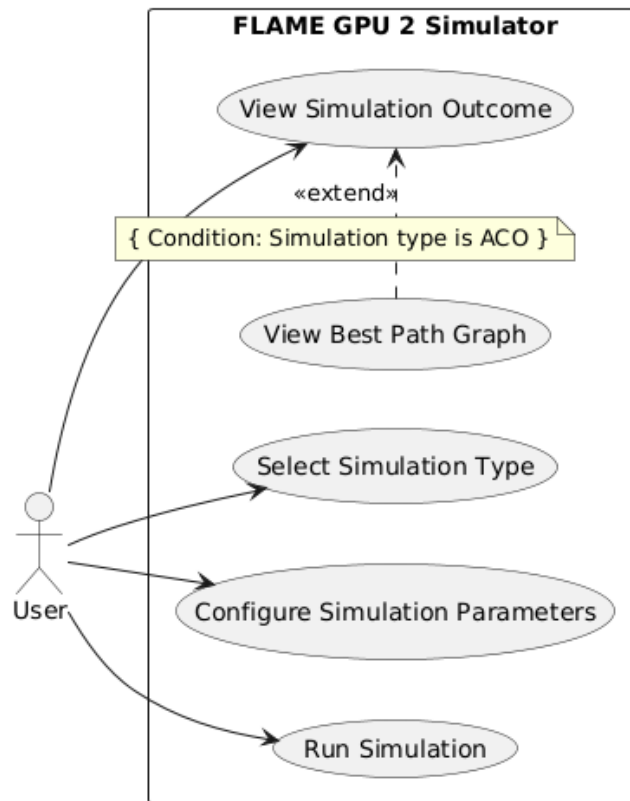


Figure 22

2. Architecture of the program

This program uses Layered Architecture, an architectural pattern that groups software components into distinct layers, each with a specific responsibility. Figure 23 shows the visualization of the Architecture of the program.

The layers are segmented into the following layers:

- User Interface (GUI) Layer: This layer handles all user interactions, such as selecting algorithms, configuring parameters and initiating the simulation.

- **Simulation Logic Layer:** This layer handles the logic of the simulation and communicating with the Simulation Engine from FLAME GPU 2.
- **Core Simulation Engine:** Provides the underlying simulation execution environment via FLAME GPU 2 library.

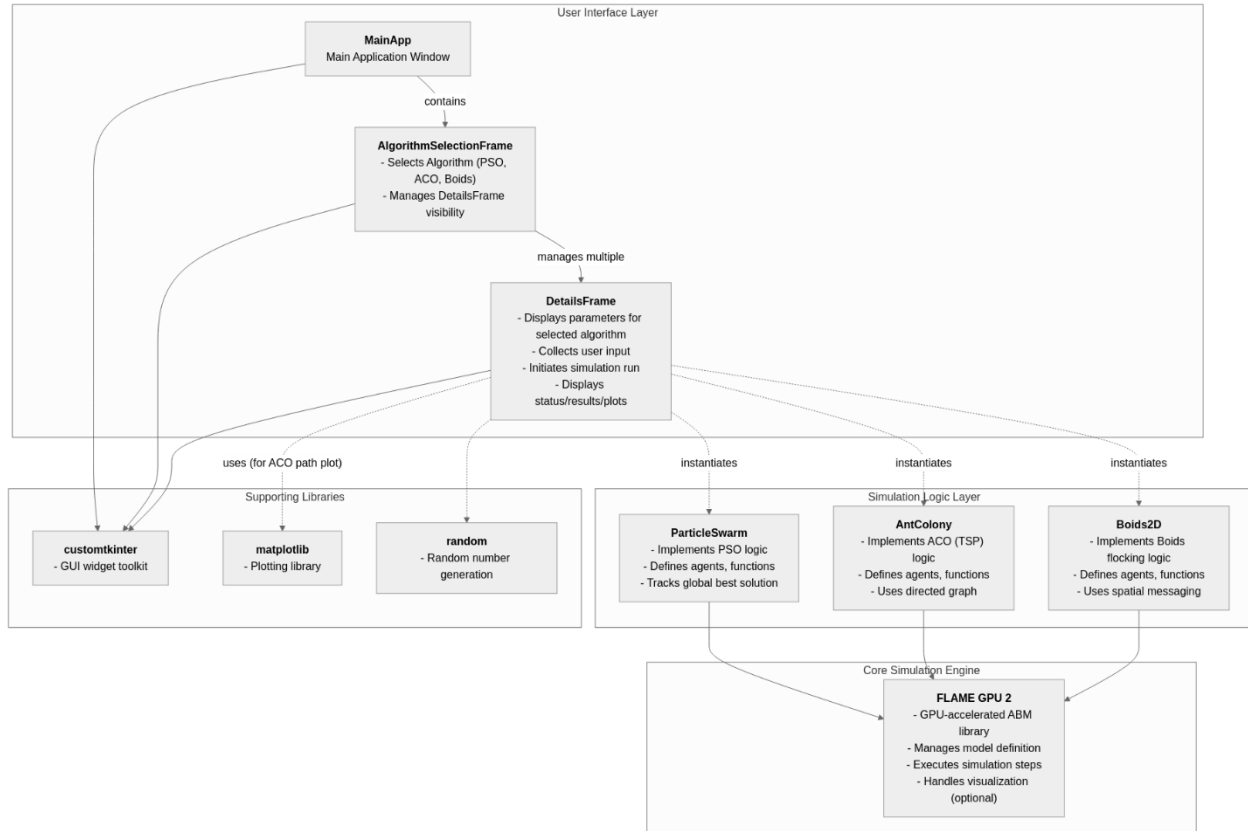


Figure 23

2.1 User Interface Layer

This layer contains the UI component of the software.

- **MainApp:** This class represents the main application window. It serves as a container for the other UI.
- **AlgorithmSelectionFrame:** This frame is contained within the MainApp. It allows the user to select between the swarm intelligence algorithm such as PSO, ACO, or Boids via a radio button. It then updates the visibility of the frame between each DetailsFrame based on the user's selection.

- **DetailsFrame:** This frame contains the parameter for the corresponding algorithm based on the selected in `AlgorithmSelectionFrame`.

2.2 Simulation Logic Layer

This layer contains the specific implementation of the swarm intelligence algorithm such as PSO, ACO and Boids, mapping C++ based scripting into optimized CUDA code in Core Simulation Engine.

- **ParticleSwarm:** This implements the Particle Swarm Optimization algorithm. It defines the particle's behavior where velocity and position updates based on personal and global bests. It evaluates the objective function and tracks the global minimum position and output by the swarm.
- **AntColony:** This implements Ant Colony Optimization algorithm to solve for the Traveling Salesperson Problem (TSP).
- **Boids2D:** This implements the Boids flocking algorithm in a 2D space. It defines Boid's agent behavior based on the three rules of separation, alignment, and cohesion. It leverages the FLAME GPU capabilities for spatial messaging to efficiently determine neighbor interactions required for these rules.

2.3 Core Simulation Engine

This layer is responsible for the execution, management, and acceleration of the agent-based models (ABMs) defined in the Simulation Logic Layer. The main component of this layer is FLAME GPU 2.

3.1 Class MainApp

Main Window class for the FLAME GPU 2 simulator.

`MainApp()` Extends `ctk.CTk`: Main application class for the FLAME GPU 2 simulator.

3.1.1 Class *AlgorithmSelectionFrame*

Frame for selecting the algorithm to run.

AlgorithmSelectionFrame(container: MainApp) Extends ctk.CTkFrame: Frame for selecting the algorithm to run.

3.1.2 Class DetailsFrame

Frame for setting the parameters of the selected algorithm.

DetailsFrame() Extends ctk.CTkFrame: Frame for setting the parameters of the selected algorithm.

change_frame(): Change the displayed parameter frame based on the selected algorithm.

3.2 Class ParticleSwarm

Particle Swarm Optimization (PSO) algorithm using FLAME GPU 2.

ParticleSwarm(num_particles: int, bound_min: float, bound_max: float, objective_function: callable, steps: int, random_seed: int, simulation_speed: int, inertia_weight: float, cognitive_coefficient: float, social_coefficient: float, function_tolerance: float, visualisation: boolean): Initialize the PSO algorithm.

run(): Runs the simulation.

3.2.1 Class update_best_cost

Check and update the best cost and position of the particles.

update_best_cost(objective_function: callable) Extends pyflamegpu.HostFunction: Initialize the update best cost function.

run(FLAMEGPU: pyflamegpu.HostAPI): Update the best cost and position of the particles.

3.2.2 class agent_exit_condition

Exit condition for the simulation when all particles have converged.

agent_exit_condition() Extends pyflamegpu.HostCondition: Initialize agent exit condition class.

run(FLAMEGPU: pyflamegpu.HostAPI): Verify that all particles have found their local best-known positions. This will stop simulation once this is complete for all particles.

3.2.3 class create_agent

Create the particles in the simulation.

create_agent(num_particles: int, bound_min: float, bound_max: float, objective_function: callable)

Extends pyflamegpu.HostFunction: Initialize Create agent class.

run(FLAMEGPU: pyflamegpu.HostAPI): Creates particle and assign each particle a random location and vector.

3.2.4 Function movement

movement(message_in: pyflamegpu.MessageNone, message_out: pyflamegpu.MessageNone): Particle movement function for the PSO algorithm.

3.3 Class Boids2D

A class to simulate a flock of Boids in 2D space using the FLAME GPU 2.

Boids2D(steps: int, random_seed: int, agent_count: int, simulation_speed: int, grid_size: int, vision_radius: float, acceleration: float, separation_radius: float, alignment_weight: float, cohesion_weight: float, separation_weight: float, max_steer_force: float, max_speed: float, visualisation: bool): Initialize the Boids2D simulation.

run(): Runs the Boids2D simulation.

3.3.1 class create_boid

A class to create Boids in the simulation.

create_boid(agent_count: int, grid_size: int) Extends pyflamegpu.HostFunction: Initialize create Boids class.

run(FLAMEGPU: pyflamegpu.HostAPI): Create a specified number of boids with a random location within the grid size.

3.3.2 Function output_position

output_position(message_in: pyflamegpu.MessageNone, message_out: pyflamegpu.MessageSpatial2D): Set output position message value for other Boids.

3.3.3 Function update_boid

update_boid(message_in: pyflamegpu.MessageSpatial2D, message_out: pyflamegpu.MessageNone): Update the Boid's position and velocity based on the rules of flocking behavior such as separation, alignment and cohesion.

3.4 Class AntColony

Ant Colony Optimization simulation on TSP using FLAMEGPU 2.

`AntColony(grid_size: int, graph_vertices: list, vertices_count: int, steps: int, random_seed: int, agent_count: int, alpha: float, beta: float, iterations: int, pheromone_evaporation: float, starting_node: int, visualisation: bool, simulation_speed: float)`: Initializes the AntColony class.
`run()`: Runs the Ant colony optimization.

3.4.1 Class *create_agent*

Create agents in the simulation.

`create_agent(agent_count : int, graph_point: list[tuple[int, int]])` Extends `pyflamegpu.HostFunction`: Initialize the create agent class.
`run(FLAMEGPU: pyflamegpu.HostAPI)`: Creates ant in the and sets its in location to the starting node.

3.4.2 Class *ant_graph_hostfn*

Create an undirected weighted graph.

`ant_graph_hostfn(graph_point: list, distance_list: list)` Extends `pyflamegpu.HostFunction`: Initialize the ant graph class.
`run(FLAMEGPU: pyflamegpu.HostAPI)`: Creates a graph in the environment.

3.4.3 Class *ant_restart*

Reset all agents value and the adjusted pheromone level. Update for best path and cost.

`ant_restart(max_iterations: int, graph_point: list)` Extends `pyflamegpu.HostFunction`: Initialize the ant restart class.
`run(FLAMEGPU: pyflamegpu.HostAPI)`: Reset all agents value and update pheromone level, best path and cost.

4. Implementation Notes

4.1 Technical Challenges and Solutions

These are several interesting challenges I have encountered while developing the program. This challenge primary stems from the Linux handles rendering graphics on a hybrid graphics system.

FLAME GPU Visualization on Linux with Hybrid Graphics

- **Challenge:** When running the simulation with visualization on a Linux system with a hybrid graphics setup where a system has both an integrated GPU from the CPU and a discrete Nvidia GPU which leads to the program crashing. By default, the system might use the integrated GPU for rendering the FLAME GPU visualization window instead of the discrete Nvidia GPU, leading to the program failing.
- **Solution:** In order to correctly render on a discrete Nvidia GPU, we have to add a environmental variable as a prefix to offload the rendering of the program to the discrete GPU. The command prefix is “`__NV_PRIME_RENDER_OFFLOAD=1 __GLX_VENDOR_LIBRARY_NAME=nvidia python main_ui.py`”.

4.2 Development Tools and Technologies

The development of the simulation software was chosen by selecting tools and frameworks that leverage their individual strengths to achieve effective outcomes.

- **Programming Language:** Python (3.12.10) was selected as the language for the development. Its interpreted nature allows for quick prototyping and modification, and Python's clear syntax and large ecosystem of available libraries provide further support.
- **Simulation Framework:** FLAME GPU 2 (version 2.0.0-rc.2) (Richmond, Chimeh, Chisholm, Heywood, & Leach, FLAME GPU (v2.0.0-rc.2), 2024) was selected due to its ability to abstract the complexity of CUDA programming and GPU optimization, allowing us to focus on specifying agent behavior and run simulation.
- **Graphical User Interface (GUI) Library:** CustomTkinter (version 5.2.2) (Schimansky, 2024) was utilized to build the application's user interface. It was chosen over the standard Tkinter library due to its modern visual appearance while still having the straightforward process in creating interactive user interface.

- Data Visualization Library: Matplotlib (version 3.10.1) (The Matplotlib Development Team, 2025) was used for generating graphical representations of the simulation result specifically to visualize the optimal paths discovered by the ACO algorithm.
- Testing Framework: Pytest (version 8.3.5) (Krekel, et al., 2004) was chosen for its simple syntax and ease of use in testing, particularly for unit and integration tests.

5. Testing Plan

This section shows the testing plan which aims to check the main component for correctness.

5.1 Unit Testing

5.1.1 PSO Test

Initialization test: Check whether the Particle Swarm Optimization can initialize properly.

Optimize Rosenbrock test: Check whether PSO is able to optimize for Rosenbrock function.

Optimize booth test: Check whether PSO is able to optimize for booth function.

Optimize sphere test: Check whether PSO is able to optimize for sphere function.

Optimize Beale test: Check whether PSO is able to optimize for Beale function.

Optimize Matyas test: Check whether PSO is able to optimize for Matyas function.

Test agent_exit_condition

Exit success condition test: Check whether the agent would exit upon all particles being done.

Exit failure condition test: Check whether the agent will continue while not all particles are done.

Test update_best_cost

Update new best cost test: Check whether the class will update to the new global best position and cost when a better value is found.

No change test: Check whether the class would keep the value the same when the cost is not better than the current best position and cost.

Test create_agents

Agent creation test: Check whether the number of agents is being created correctly.

5.1.2 ACO Test

Initialization test: Check whether the Ant Colony Optimization can initialize properly.

Calculate distance test: Check whether the calculated distance function is calculated correctly.

Calculate distance matrix test: Check whether the edges list is being calculated correctly.

Test agent_exit_condition

Exit success condition test: Check whether the agent would exit upon all ants being done.

Exit failure condition test: Check whether the agent will continue while not all ants are done.

5.1.3 Boids Test

Initialization test: Check whether the Boids can initialize properly.

Test create_boid

Boids creation test: Check whether the number of Boids is being created correctly.

5.2 Integration Testing

Run PSO from Main UI: Able to run PSO from the main UI.

Run ACO from Main UI: Able to run ACO from the main UI.

Run Boids from Main UI: Able to run Boids from the main UI.

PSO Run test: Check whether the Particle Swarm Optimization simulation can run.

ACO Run test: Check whether the Ant Colony Optimization simulation can run.

Boids Run test: Check whether the Boids simulation can run.

5.3 Performance Testing

This performance test was run on Asus ROG Strix G18 Laptop with i9-13989HX and Nvidia 4060 Mobile.

In each test, 20 runs were performed for each point of the graph in order to minimize errors and improve the reliability of the data points.

5.3.1 Boids Performance

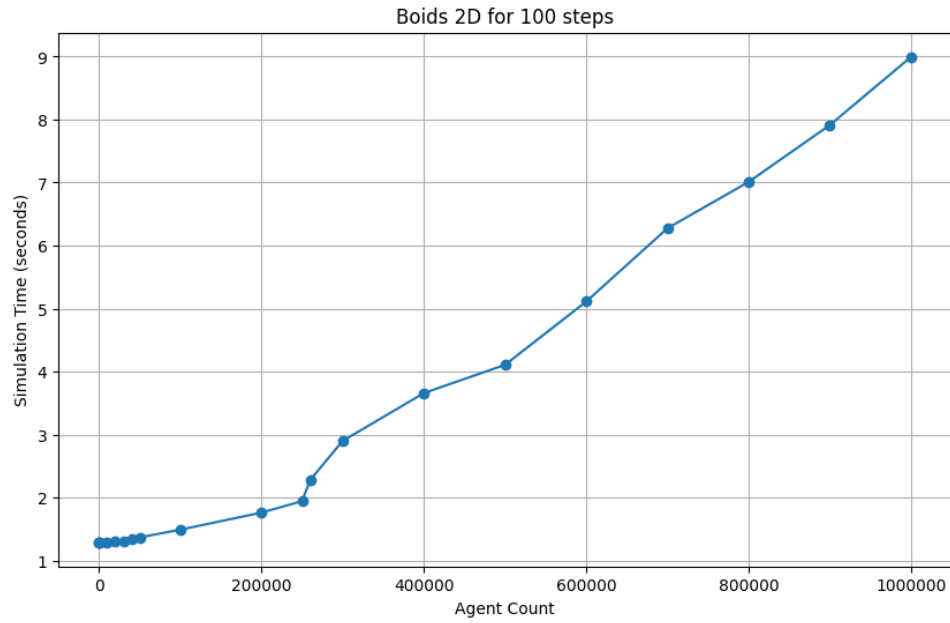


Figure 24

Figure 24 shows the performance test of Boids 2D, we observed that we are able to scale up to 200,000 Boids without experiencing any notable performance degradation. After 300,000 Boids, the simulation slows down linearly. However, a noticeable slowdown can be observed after 260,000 Boids as seen in the Figure 25.

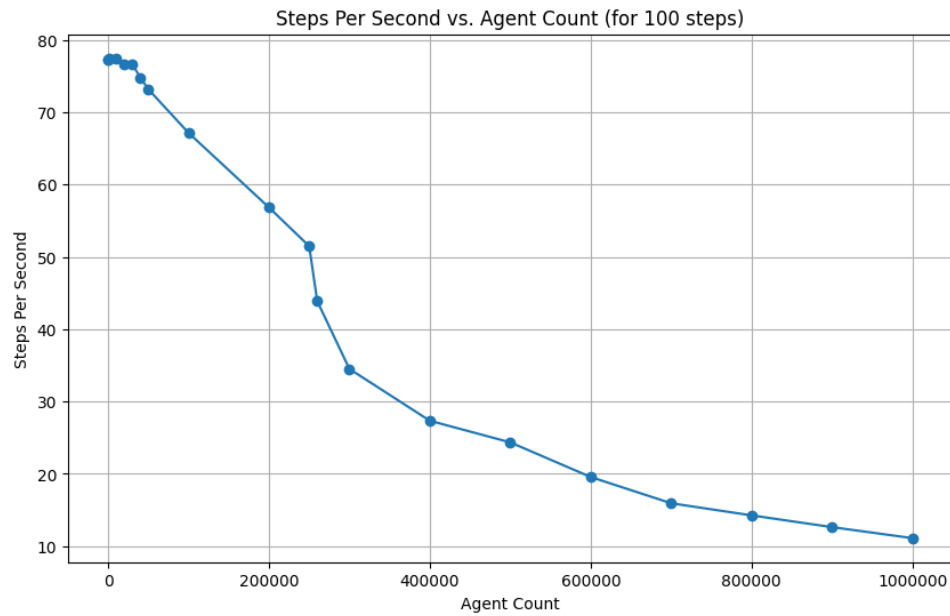


Figure 25

5.3.2 ACO Performance

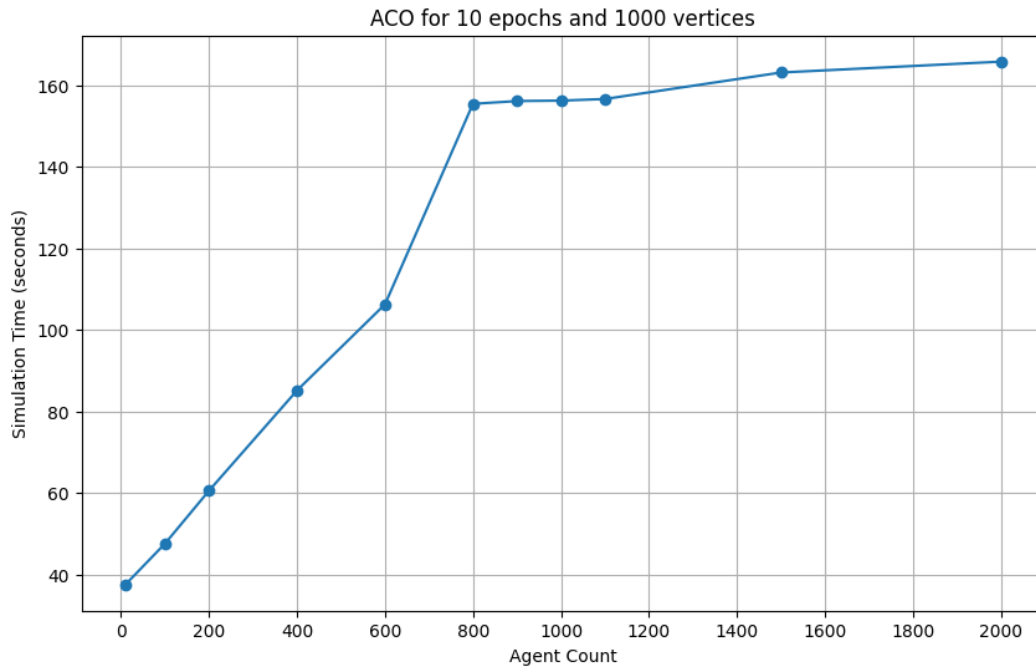


Figure 26

Figure 26 shows the performance test of ACO, we observed that the simulation time increases with the number of agents before we hit the saturation point after approximately 800 agents. After that point, the simulation time barely increases as more ants are added.

6. Testing

6.1 Unit Testing Results

6.1.1 PSO Test Results:

Initialization Test: **Passed**. PSO initialized correctly.

Optimize Rosenbrock Test: **Passed**. PSO successfully optimized the Rosenbrock function within expected parameters.

Optimize Booth Test: **Passed**. PSO successfully optimized the Booth function.

Optimize Sphere Test: **Passed**. PSO successfully optimized the Sphere function.

Optimize Beale Test: **Passed**. PSO successfully optimized the Beale function.

Optimize Matyas Test: **Passed**. PSO successfully optimized the Matyas function.

Test agent_exit_condition: Both success and failure condition tests **passed**, confirming correct agent termination logic.

Test update_best_cost: Both update and no-change tests **passed**, verifying correct global best tracking.

Test create_agents: **Passed**. The specified number of agents was created correctly.

6.1.2 ACO Test Results:

Initialization Test: **Passed**. ACO initialized correctly.

Calculate Distance Test: **Passed**. Distance calculations were verified as accurate.

Calculate Distance Matrix Test: **Passed**. The distance matrix (edges list) was generated correctly.

Test agent_exit_condition: Both success and failure condition tests **passed**, confirming correct ant termination logic.

6.1.3 Boids Test Results:

Initialization Test: **Passed**. Boids simulation initialized correctly.

Test create_boid: **Passed**. The specified number of Boids was created correctly.

6.3 Integration Testing Results

All planned integration tests passed successfully, confirming that the algorithms are correctly integrated with the main application UI and can be executed as intended.

Run PSO from Main UI: **Passed**. The PSO simulation was successfully launched and run via the main UI.

Run ACO from Main UI: **Passed**. The ACO simulation was successfully launched and run via the main UI.

Run Boids from Main UI: **Passed**. The Boids simulation was successfully launched and run via the main UI.

PSO Run Test: **Passed**. The PSO simulation executed completely without runtime errors.

ACO Run Test: **Passed**. The ACO simulation executed completely without runtime errors.

Boids Run Test: **Passed**. The Boids simulation executed completely without runtime errors.

Summary

To summarize, this thesis demonstrates the potential of using GPU acceleration to simulate swarm intelligence using FLAME GPU 2 as the framework.

FLAME GPU 2 enables GPU-accelerated agent-based modeling. The key feature of FLAME GPU 2 is the mapping it provides for the agent function to optimize CUDA code. This leverages the parallel processing power of GPU to accomplish high performance and scalability, enabling simulations of a significantly larger size than might be feasible otherwise.

The program we have built was designed to be a user-friendly desktop application by using CustomTkinter modern with selection of swarm intelligence algorithms such as Particle Swarm Optimization, Ant Colony Optimization, and Boids.

The program is packed with several distinct swarm intelligence algorithms along with a variety of adjustable parameters for each algorithm. Furthermore, users are able to visualize each algorithm working in real time. From being able to view PSO optimizing for an objective function in real-time to the ability to change a variety of parameters of Boids as emergent behavior occurs in the simulation in real-time and seeing the best path found by the colony from ACO. The program also comes with Reproducibility in mind with a configurable random seed for practical and reliable research purposes.

References

- Dorigo, M., & Gambardella, L. M. (1997). Ant colony system: a cooperative learning approach to the traveling salesman problem. *IEEE Transactions on Evolutionary Computation*, 1(1), 53-66. doi:10.1109/4235.585892
- Kennedy, J., & Eberhart, R. (1995). Particle swarm optimization. *Proceedings of ICNN'95 - International Conference on Neural Networks*, (pp. 1942-1948). doi:10.1109/ICNN.1995.488968
- Krekel, H., Oliveira, B., Pfannschmidt, R., Bruynooghe, F., Laughner, B., & Bruhin, F. (2004). *pytest* 8.3.5. Retrieved from <https://github.com/pytest-dev/pytest>
- Reynolds, C. W. (1987). Flocks, herds and schools: A distributed behavioral model. *Proceedings of the 14th Annual Conference on Computer Graphics and Interactive Techniques* (pp. 25–34). New York: Association for Computing Machinery. doi:10.1145/37401.37406
- Richmond, P., Chimeh, M. K., Chisholm, R., Heywood, P., & Leach, M. (2023). FLAME GPU 2: A framework for flexible and performant agent based simulation on GPUs. *Software: Practice and Experience*, 53(8), 1659-1680. doi:<https://doi.org/10.1002/spe.3207>
- Richmond, P., Chimeh, M. K., Chisholm, R., Heywood, P., & Leach, M. (2024). *FLAME GPU* (v2.0.0-rc.2). Zenodo. doi:10.5281/zenodo.14204752
- Schimansky, T. (2024). *CustomTkinter* (5.2.2. Retrieved from <https://github.com/TomSchimansky/CustomTkinter>
- Shi, Y., & Eberhart, R. (1998). A modified particle swarm optimizer. *1998 IEEE International Conference on Evolutionary Computation Proceedings. IEEE World Congress on Computational Intelligence*, (pp. 69-73). doi:10.1109/ICEC.1998.699146
- The Matplotlib Development Team. (2025). *Matplotlib: Visualization with Python* (v3.10.1). Zenodo. doi:10.5281/zenodo.14940554